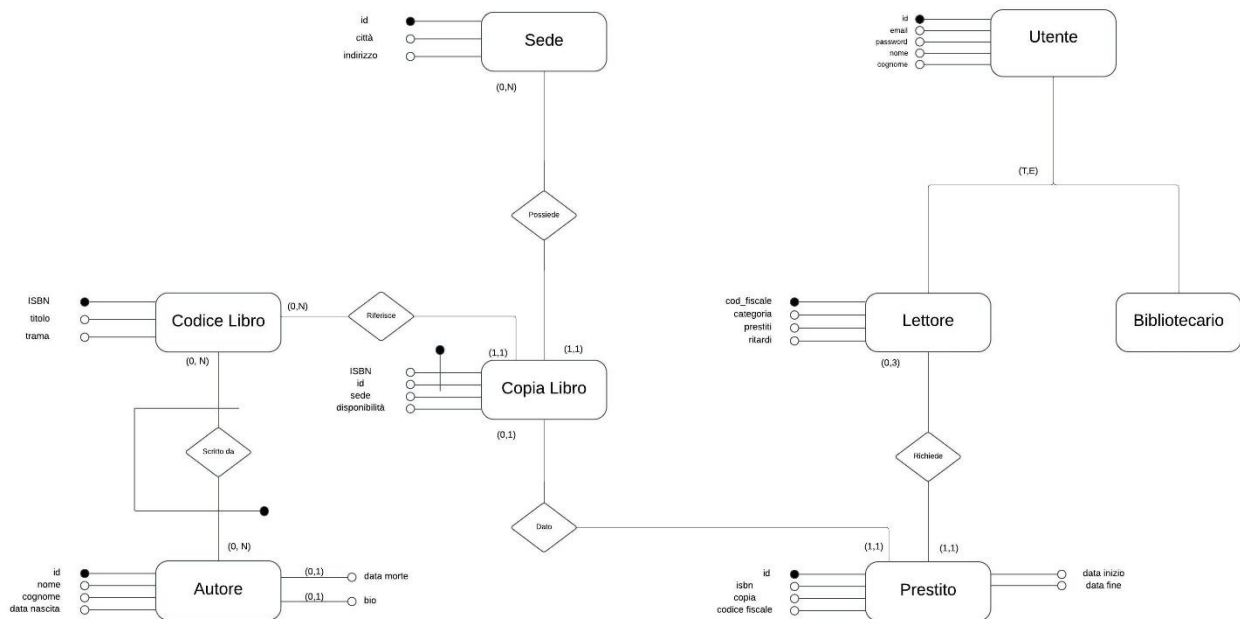


DOCUMENTAZIONE TECNICA

PROGETTO BASI DI DATI

Kevin Softic - 975501

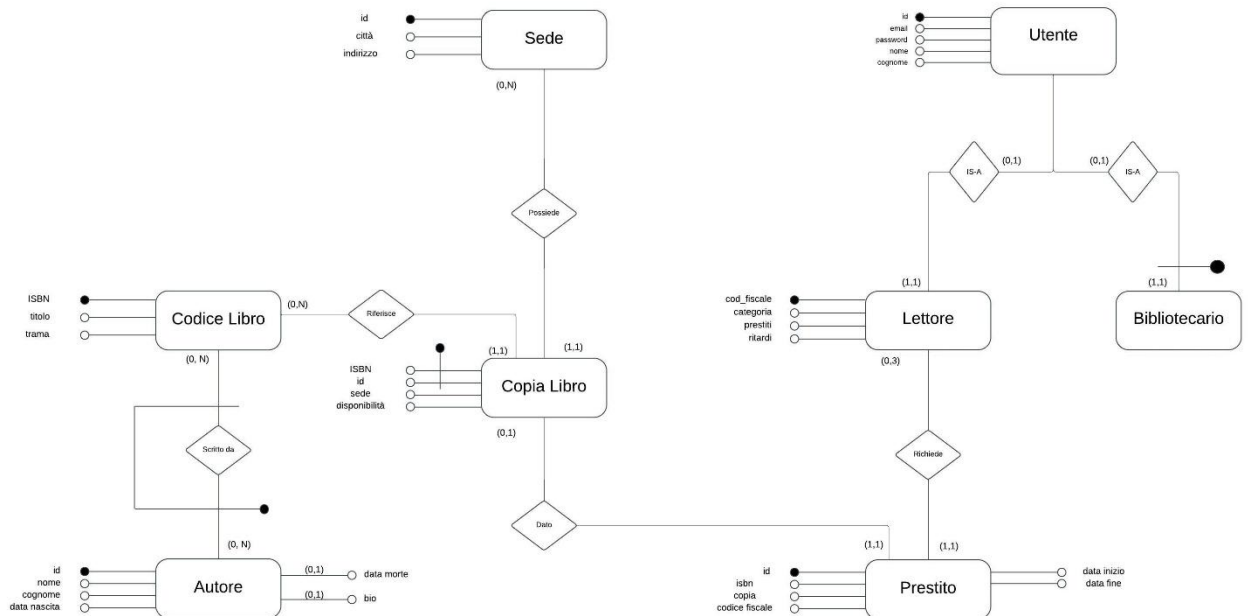
Schema concettuale (ER) della base di dati



Considerazioni sulle scelte progettuali:

- **gerarchia di generalizzazione per gli utenti**: tutti gli utenti sono dotati di un login con credenziali quali email e password, ho così deciso di accorpare queste informazioni insieme ad alcune sempre presenti quali nome e cognome nella entità padre **utenti**, avendo come chiave primaria l'id, il codice fiscale è chiave ma solo per il **lettore**, che possiede informazioni aggiuntive come categoria, numero di prestiti in corso e numero di ritardi totali. Le entità **bibliotecario** non possiedono nessuna informazione aggiuntiva, risultando come tabelle con solamente la chiave esterna per estrarre un **sottoinsieme di utenti con compiti e diritti specifici**, ma si potrebbe aggiungere facilmente informazioni come sede di lavoro, numero telefonico, ecc...
- **ogni lettore può avere al massimo 3 oppure 5 libri in prestito contemporaneamente**: nello schema ER questo concetto viene rappresentato da un vincolo di cardinalità tra lettore e prestito (0,3) (nel caso base), ossia un lettore può avere al massimo 3 prestiti, e la copia che viene prestata è singola.
- **gestione degli autori ed i loro relativi libri**: un autore può scrivere diversi libri, così come un libro può essere scritto dalla collaborazioni di multipli autori, questo comportamento viene rappresentato da una relazione tra autore e codice del libro di cardinalità (0, N), in quanto ho considerato plausibile avere autori con libri pubblicati ma non presenti nella biblioteca o avere libri ma senza particolari informazioni sull'autore.

Ristrutturazione schema logico



Ho deciso di mantenere tutte le entità con associazioni 1:1 per rappresentare il legame IS-A, cioè che ogni istanza delle entità figlie coincide con una e una sola istanza dell'entità padre.

Schema logico (relazionale) della base di dati

Ecco lo schema prodotto con la traduzione dallo schema ER:

Utenti(id, email, password, nome, cognome)

Bibliotecari(*Utenti*)

Lettori(cod_fiscale, *Utenti*, categoria, prestiti, ritardi)

Sedi(id, città, indirizzo)

Cod_Libro(isbn, titolo, trama)

Libri(Cod_Libro, id, Sede, disponibilità)

Autori(id, nome, cognome, data_nascita, data_morte, bio)

Scritto_da(Cod_Libro, Autori)

Prestiti(id, Cod_Libro, Libri, Lettori, data_inizio, data_fine)

Appunti sulle motivazioni di questa traduzione:

- per il prestito di copie del libro le associazioni 1:1 sono state tradotte mettendo nell'entità prestiti una chiave esterna
- per il prestito di copie ad uno studente le associazioni (in questo caso 1:N) mettendo nell'entità che ha cardinalità max = 1 la chiave esterna

- la relazione possiede si traduce come una 1:N con chiave esterna in libri
- per quanto riguarda le copie di un libro le associazioni 1:N sono state tradotte mettendo nell'entità libri la chiave esterna
- la relazione scritto da si traduce come una normale relazione (N:M) molti a molti

Scelte implementative e gestione dei casi limite

Scelte implementative

Per quanto riguarda i codici univoci ho deciso di utilizzare il tipo di dato **UUID**, cioè un tipo di dato unico ma non incrementale, risultando più sicuro rispetto ad un **serial**, per esempio.

Ho deciso di utilizzare il tipo **DATE** ritenendolo sia un tipo appropriato sia utile per i vari confronti.

Per quanto riguarda il lettore ho anche utilizzato degli **enum** per evitare check ovvi.

Le password degli utenti non sono *ovviamente* salvate in chiaro, ma vengono "**hashate**" e "**saltate**" all'inserimento, attraverso l'utilizzo della funzione crypt dell'estensione di postgres pgcrypto

```
CREATE TABLE utenti(
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  ...
);

CREATE TABLE autori(
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  data_nascita date NOT NULL,
  ...
);

CREATE TYPE TIPO_LETTORE AS enum ('Base', 'Premium');
...

...
INSERT INTO utenti(email, password, nome, cognome)
VALUES (_email, crypt(_password, gen_salt('bf')), _nome, _cognome) RETURNING id INTO _id;
...
```

Controllo per un nuovo prestito

Questo controllo l'ho realizzato inserendo un trigger su insert nella tabella prestiti, cioè quando vado a creare un nuovo prestito di un libro, eseguendo vari controlli:

- controllo che il libro sia effettivamente disponibile
- controllo che il lettore possa effettivamente effettuare un prestito, cioè che il numero di prestiti attivi sia minore del massimo ammesso per la sua categoria e che il numero di consegne in ritardo effettuate sia minore di 5.
- controllo che la data di fine prestito non sia prima della data d'inizio

```

CREATE OR REPLACE FUNCTION check_numeroprestiti() RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    _in_corso integer;
    _in_ritardo integer;
    _counter smallint;
    _tipo TIPO_LETTORE;
    _disponibile bool;
BEGIN
    SET search_path TO unibib;

    --confronto il numero di ritardi dalla tabella lettori
    SELECT l.ritardi, l.categoria INTO _counter, _tipo
    FROM lettori AS l
    WHERE l.cod_fiscale = NEW.cod_fiscale;

    --controllo la disponibilità di un libro
    SELECT li.disponibilita INTO _disponibile
    FROM libri AS li
    WHERE li.isbn = NEW.isbn AND li.id = NEW.copia_id;

    IF NEW.data_inizio > NEW.data_fine THEN
        RAISE EXCEPTION 'La data di fine prestito non può essere prima della data di inizio';
    END IF;

    IF _disponibile = false THEN
        RAISE EXCEPTION 'Il libro non è disponibile';
    END IF;

    SELECT COUNT(*) INTO _in_ritardo
    FROM prestiti AS p
    WHERE p.cod_fiscale = NEW.cod_fiscale AND p.data_fine < CURRENT_DATE;

    IF (_counter + _in_ritardo) > 4 THEN
        RAISE EXCEPTION 'Raggiunto il massimo numero di consegne in ritardo, contattare la segreteria.';
    END IF;

    -- controllo il numero di prestiti in corso del lettore
    SELECT COUNT(*) INTO _in_corso
    FROM prestiti AS p
    WHERE p.cod_fiscale = NEW.cod_fiscale;

    IF _tipo = 'Base' AND _in_corso > 2 THEN
        RAISE EXCEPTION 'Raggiunto il numero massimo di prestiti contemporanei per il piano base, passare al piano premium';
    END IF;

    IF _tipo = 'Premium' AND _in_corso > 4 THEN
        RAISE EXCEPTION 'Raggiunto il numero massimo di prestiti contemporanei permesso';
    END IF;

    RETURN NEW;
END;
$$;

```

Controllo di proroga prestito

Nel caso invece di un update nella tabella prestiti, cioè se si vuole aggiornare il prestito per fare una proroga sulla data di fine prestito, controllo che il prestito non sia già in ritardo e che la data selezionata non sia nel passato.

```

CREATE OR REPLACE FUNCTION check_data_prestito() RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    _data_inizio date;
    _data_fine date;
BEGIN
    SET search_path TO unbib;

    --confronto la data di fine non sia prima della data di inizio oppure della data odierna
    SELECT data_inizio,data_fine INTO _data_inizio, _data_fine
    FROM prestiti
    WHERE id = NEW.id;

    IF _data_fine < CURRENT_DATE THEN
        RAISE EXCEPTION 'Non si può cambiare la data di un prestito già in ritardo';
    END IF;

    IF NEW.data_fine < _data_inizio OR NEW.data_fine < CURRENT_DATE THEN
        RAISE EXCEPTION 'La data di fine non può essere nel passato';
    END IF;

    RETURN NEW;
END
$$;

--trigger che controlla la data di fine prestito nel caso si decida di modificarla (es. proroga)
--considerando che la data di fine non può essere nel passato o prima della data di inizio del prestito
CREATE OR REPLACE TRIGGER u_check_prestito
BEFORE UPDATE ON prestiti
FOR EACH ROW
EXECUTE PROCEDURE check_data_prestito();

```

Controllo per nuovi autori

Siccome gli autori possono non avere una data di morte nel caso siano ancora vivi, controllo inserendo un trigger su insert e update nella tabella autori, quando inserisco o aggiorno un nuovo autore, che nel caso la data di morte sia inserita essa non sia prima della data di nascita oppure che non sia in una data che non è ancora avvenuta.

```

CREATE OR REPLACE FUNCTION check_morte_autore() RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    _nascita date;
BEGIN
    SET search_path TO unbib;
    -- controllo che se la data di morte è stata inserita, essa non sia prima della data di nascita
    SELECT a.data_nascita INTO _nascita
    FROM autori AS a
    WHERE a.id = NEW.id;

    IF NEW.data_morte IS NOT NULL AND (_nascita > NEW.data_morte OR NEW.data_morte > CURRENT_DATE) THEN
        RAISE EXCEPTION 'La data di morte non può essere prima della data di nascita o nel futuro';
    END IF;

    RETURN NEW;
END;
$$;

--trigger che controlla la data di morte quando la si inserisce per un autore
--controlla soprattutto che non sia prima della data di nascita o nel futuro
CREATE OR REPLACE TRIGGER i_u_check_autore
BEFORE INSERT OR UPDATE ON autori
FOR EACH ROW
EXECUTE PROCEDURE check_morte_autore();

```

Eliminazione di libri non più mantenuti

Controllo realizzato inserendo un trigger su delete nella tabella cod_libro, cioè l'eliminazione di libri non più mantenuti dal sistema, controllo che non ci siano prestiti in corso di neanche una copia del libro in questione (una versione analoga esiste anche nel caso si elimini solo una copia di

un libro).

```
CREATE OR REPLACE FUNCTION check_fine_prestito() RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
    DECLARE
        _count integer;
    BEGIN
        SET search_path TO unibib;
        --controlla che tutti i prestiti del libro siano finiti
        SELECT COUNT(*) INTO _count
        FROM prestiti AS p
        WHERE p.isbn = OLD.isbn;

        IF _count > 0 THEN
            RAISE EXCEPTION 'Non è possibile eliminare libri che sono ancora in prestito';
        END IF;

        RETURN OLD;
    END;
$$;

--trigger che controlla se tutti i libri sono nelle sedi per poter eliminare quel libro e le relative copie
CREATE OR REPLACE TRIGGER d_libro
BEFORE DELETE ON cod_libro
FOR EACH ROW
EXECUTE PROCEDURE check_fine_prestito();
```

Eliminazione di lettori

In modo analogo all'eliminazione di libri controllo che il lettore che si vuole eliminare non abbia dei prestiti in corso.

```
CREATE OR REPLACE FUNCTION check_nessun_prestito() RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
    DECLARE
        _count integer;
    BEGIN
        SET search_path TO unibib;
        --controlla che l'utente non abbia prestiti in corso
        SELECT COUNT(*) INTO _count
        FROM prestiti AS p
        WHERE p.cod_fiscale = OLD.cod_fiscale;

        IF _count > 0 THEN
            RAISE EXCEPTION 'Impossibile eliminare utenti con prestiti in corso';
        END IF;

        RETURN OLD;
    END;
$$;

--trigger che controlla che il lettore non abbia prestiti in corso per poterlo eliminare
CREATE OR REPLACE TRIGGER d_lettore
BEFORE DELETE ON lettori
FOR EACH ROW
EXECUTE PROCEDURE check_nessun_prestito();
```

Descrizioni delle funzioni realizzate

Procedure

Elenco procedure:

- add_reader: crea un nuovo lettore (ed il relativo utente) dato codice fiscale, email, password, nome e cognome.

- `edit_reader`: modifica un lettore (ed il relativo utente) dato il suo codice fiscale, email, nome, cognome, categoria e numero ritardi.
- `delete_reader`: elimina un lettore dato il suo codice fiscale.
- `add_librarian`: aggiunge un bibliotecario (ed il relativo utente) dato email, password, nome, cognome.
- `edit_password_as_admin`: modifica la password di un utente dato solo il suo id e la nuova password.
- `edit_password_user`: modifica la password di un utente dato il suo id, la vecchia password e la nuova password.
- `add_library`: crea una nuova sede dato città ed indirizzo.
- `edit_library`: modifica una sede dato la sua id, città ed indirizzo.
- `add_book`: crea un nuovo libro (ed una prima copia) dato isbn, sede, titolo, trama (ed eventuali autori).
- `edit_book`: modifica un libro dato il suo isbn, titolo, trama (ed eventuali autori).
- `delete_book`: elimina un libro (e tutte le sue copie) dato il suo codice isbn.
- `add_copy`: crea una copia di un libro dato codice isbn e sede.
- `edit_copy`: modifica una copia di un libro dato il codice isbn, il suo codice id e sede.
- `delete_copy`: elimina una copia di un libro dato il codice isbn e il suo codice id.
- `add_writer`: crea un autore dato nome, cognome, data di nascita (ed eventuali data di morte, biografia, libri).
- `edit_writer`: modifica un autore dato il suo id, nome, cognome, data di nascita (ed eventuali data di morte, biografia, libri).
- `delete_writer`: elimina un autore dato il suo id.
- `add_rent`: crea un prestito dato il codice isbn e l'id della copia, il codice fiscale del lettore, data di inizio e data di fine.
- `edit_rent`: modifica un prestito dato il suo codice id, data di fine e data di consegna.

Funzioni

Elenco funzioni:

- `login`: Verifica il login di un utente in base ad email e password.
- `get_reader`: restituisce un lettore in base al suo codice fiscale.
- `get_all_readers`: restituisce tutti i lettori.
- `get_librarian`: restituisce un bibliotecario in base al suo id.
- `get_isbn_book`: restituisce un libro in base al suo codice isbn (ed eventualmente una sede).
- `get_title_book`: restituisce un libro in base al suo titolo (ed eventualmente una sede).
- `get_all_books`: restituisce tutti i libri.
- `get_writer`: restituisce un autore in base al suo id.
- `get_all_writers`: restituisce tutti gli autori.
- `get_writers_of_book`: restituisce tutti gli autori di un libro in base al suo codice isbn.
- `get_books_of_writer`: restituisce tutti i libri di un autore in base al suo id.
- `get_city_library`: restituisce le sedi data una città.

- `get_all_libraries`: restituisce tutte le sedi.
- `get_active_rent`: restituisce tutti i prestiti.
- `get_active_rent_reader`: restituisce i prestiti di un lettore dato il suo codice fiscale.
- `get_statistics`: restituisce le statistiche di una sede dato il suo id.
- `get_report`: restituisce i report sui ritardi di una sede dato il suo id.

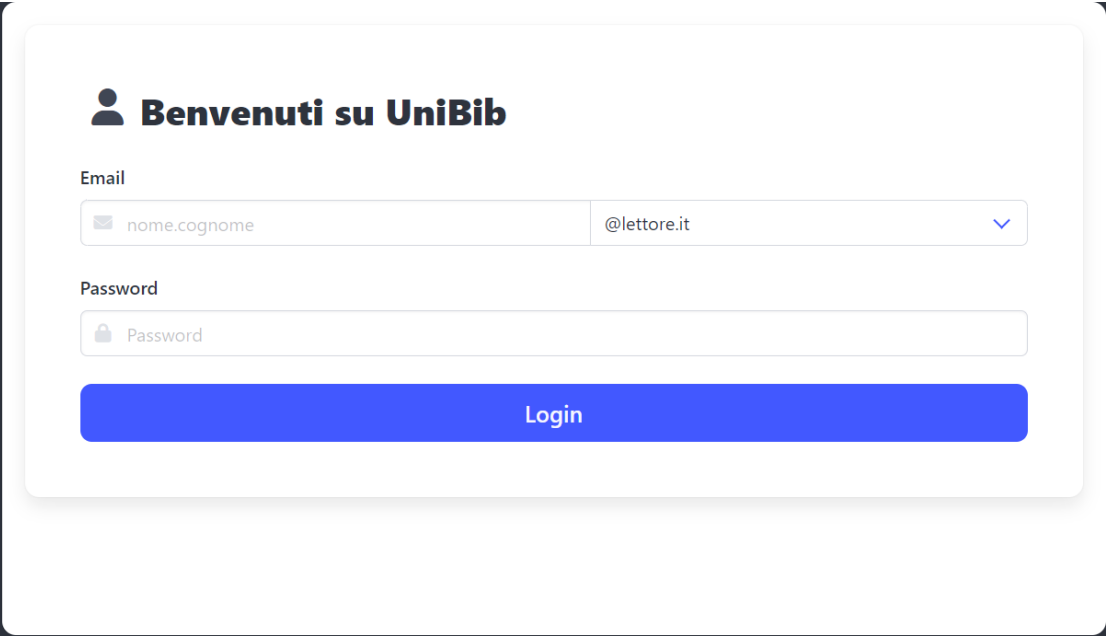
Trigger

Elenco trigger:

- `i_check_prestito`: controlla, alla creazione di un nuovo prestito, che le date, la copia, e il lettore siano adeguati al prestito.
- `u_check_prestito`: controlla all'aggiornamento di un prestito che la data
- `i_u_check_autore`: controlla, alla creazione o all'aggiornamento di un nuovo autore, che la data di morte, se inserita, sia dopo la data di nascita e prima di domani.
- `d_libro`: controlla, all'eliminazione di un libro, che tutte le copie non siano in prestito.
- `d_copia`: controlla, all'eliminazione di una copia, che la copia non sia in prestito.
- `d_lettore`: controlla, all'eliminazione di un lettore, che il lettore non abbia nessun prestito in corso.

Prove di funzionamento

Una volta andati nel proprio browser a <https://localhost/unibibweb/index.php> ci ritroveremo nella schermata di login. Nel menu a tendina si può scegliere il tipo di utente per il login.

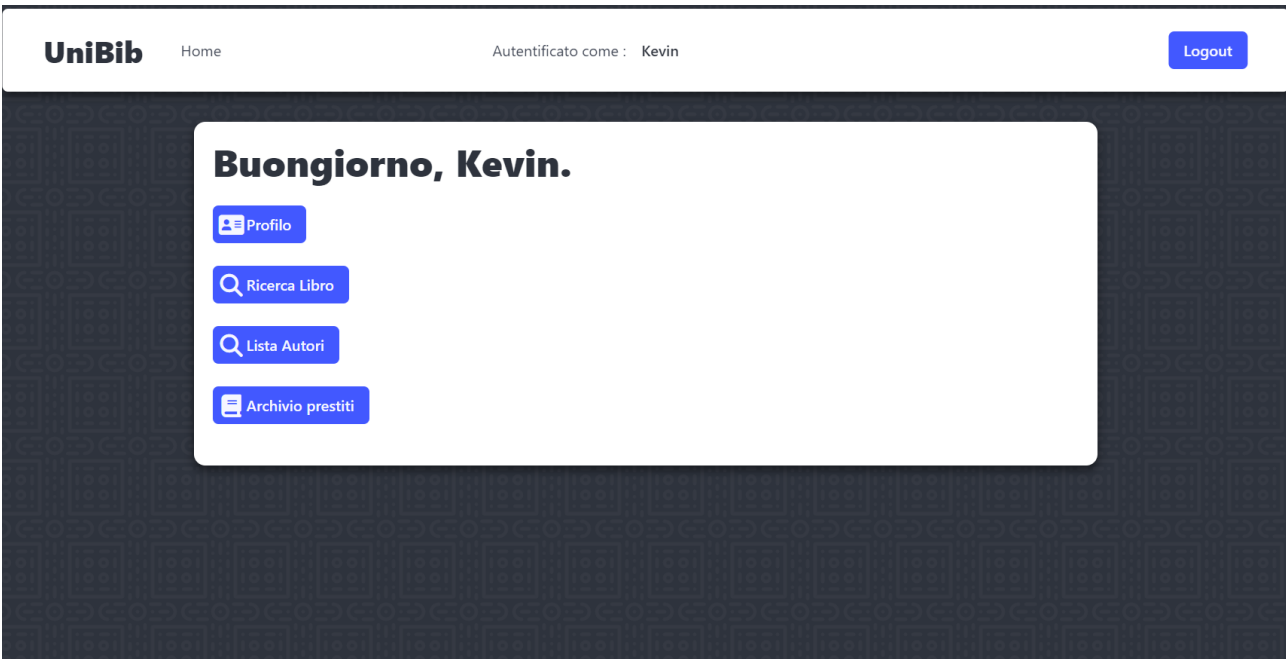


The login form is titled "Benvenuti su UniBib" and features a user icon. It includes an "Email" section with a text input field containing "nome.cognome" and a dropdown menu set to "@lettore.it". Below this is a "Password" section with a text input field containing "Password". A prominent blue "Login" button is positioned at the bottom of the form. The entire form is set against a dark blue background with a subtle geometric pattern.

Built by [Kevin Softic](#).

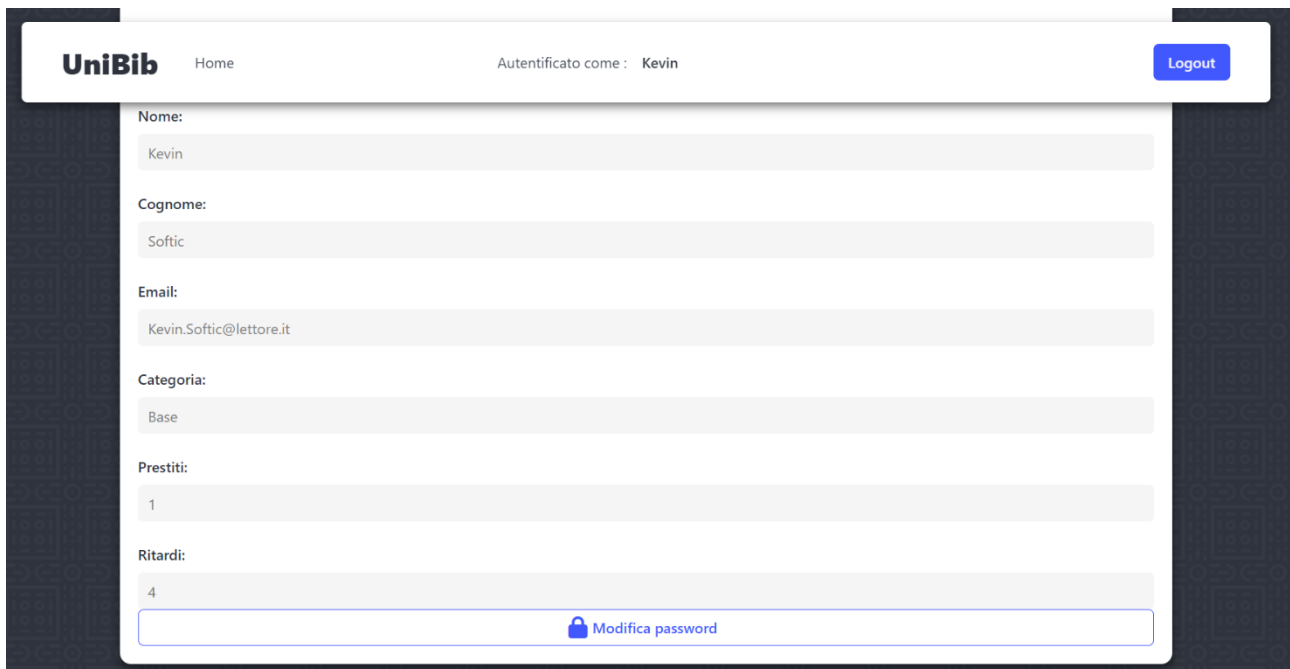
Lettore

Logghiamo con le credenziali di un lettore. Così da trovarci nella homepage dell'utente:



The user homepage for "Kevin" features a top navigation bar with the "UniBib" logo, a "Home" link, and the text "Autenticato come: Kevin" next to a "Logout" button. The main content area has a dark blue background with a geometric pattern. A white card displays the greeting "Buongiorno, Kevin." followed by four blue buttons: "Profilo", "Ricerca Libro", "Lista Autori", and "Archivio prestiti".

Nel riquadro abbiamo le varie operazioni che un lettore può fare. Ad esempio se seleziono l'opzione "Profilo" è possibile osservare le proprie informazioni e credenziali:

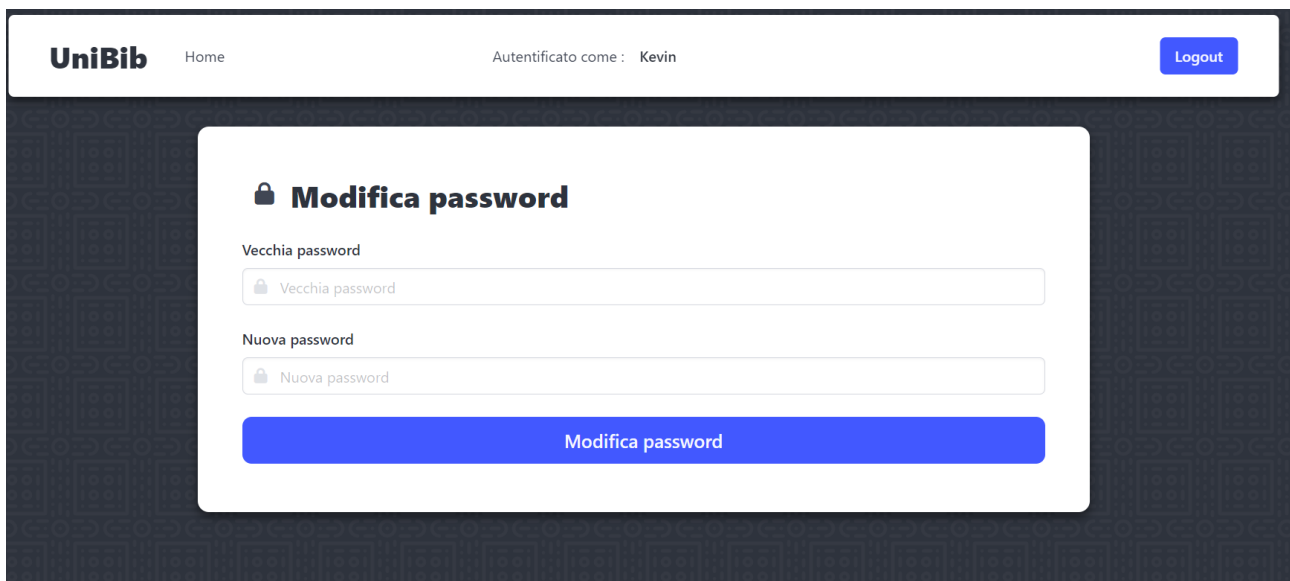


The screenshot shows the UniBib user profile page. At the top, there is a header with the UniBib logo, a "Home" link, and the text "Autenticato come : Kevin". A "Logout" button is in the top right corner. The main content area displays the user's profile information in a form-like layout:

- Nome:** Kevin
- Cognome:** Softic
- Email:** Kevin.Softic@lettore.it
- Categoria:** Base
- Prestiti:** 1
- Ritardi:** 4

At the bottom of the profile section, there is a button with a lock icon and the text "Modifica password".

e premendo il pulsante cambia password è possibile modificarla:



The screenshot shows the UniBib "Modifica password" page. The header is identical to the previous screenshot. The main content area features a white box with the title "Modifica password" and a lock icon. Below the title, there are two input fields:

- Vecchia password:** A text input field with a lock icon and the placeholder text "Vecchia password".
- Nuova password:** A text input field with a lock icon and the placeholder text "Nuova password".

Below these fields is a large blue button with the text "Modifica password".

Selezionando invece l'opzione "Ricerca Libro" abbiamo la possibilità di cercare libri nel sistema, la ricerca può essere fatta tramite codice ISBN oppure tramite il titolo del libro, con il menu a tendina si può selezionare il tipo di ricerca da effettuare, in più c'è la possibilità di specificare una determinata sede selezionandola dal menu a tendina di tutte le sedi disponibili:

UniBib

Home

Autenticato come : Kevin

Logout

Cerca un libro

Cerca per

ISBN

ISBN

Titolo

Inserisci ISBN o titolo

Sede

Seleziona una sede (opzionale)

Cerca

Dopodiché si riceverà una tabella dalla quale, oltre ad ottenere le informazioni riguardo la sede e la disponibilità delle copie del libro ricercato, sarà possibile fare la richiesta di prestito se risulta disponibile:

UniBib

Home

Autenticato come : Kevin

Logout

Ricerca Libro

ISBN	Titolo	Città	Indirizzo	Disponibilità	Richiedi prestito
IF200S02	Seconda Prova	Milano	Corso Italia 20		Prestito
IF200S02	Seconda Prova	Milano	Via Celoria 18		Prestito
IF200S02	Seconda Prova	Milano	Corso Italia 20		Prestito

Nel caso un libro non sia presente nella sede selezionata, si verrà informati di ciò e inoltre verrà mostrata una lista di copie del libro ricercato in tutte le sedi che lo hanno in gestione:

UniBib

Home

Autenticato come : Kevin

Logout

Nessun risultato trovato per la sede selezionata... Ecco una lista di risultati senza una sede specificata

Ricerca Libro

ISBN	Titolo	Città	Indirizzo	Disponibilità	Richiedi prestito
IF200S04	Quarta Prova	Milano	Via Celoria 18		Non disponibile

Abbiamo inoltre l'opzione "Lista Autori" che permette data una lista di autori di visualizzare il loro profilo

Lista Autori

Nome	Cognome	Profilo
Dante	Aligo	Profilo
Dante	Alighieri	Profilo
Dante	Senza Morte	Profilo

Dal loro profilo, oltre a visualizzare i loro dati personali, si può visualizzare la lista dei libri scritti gestiti dal sistema e cliccandoci sopra permette di passare alla pagina di ricerca effettuata per richiedere il prestito:

UniBib

[Home](#)

Autenticato come : Kevin

[Logout](#)

Nome

Dante

Cognome

Aligo

Data di nascita

15/10/1500

Data di morte

01/08/2024

Biografia

Ciao

Libri

IF200S04-Quarta Prova

IF200S03-Terza Prova

IF200S02-Seconda Prova

Infine, come ultima opzione, “Archivio prestiti” permette di vedere la lista dei prestiti che abbiamo in corso, con il relativo periodo di prestito:

UniBib

[Home](#)

Autenticato come : Kevin

[Logout](#)

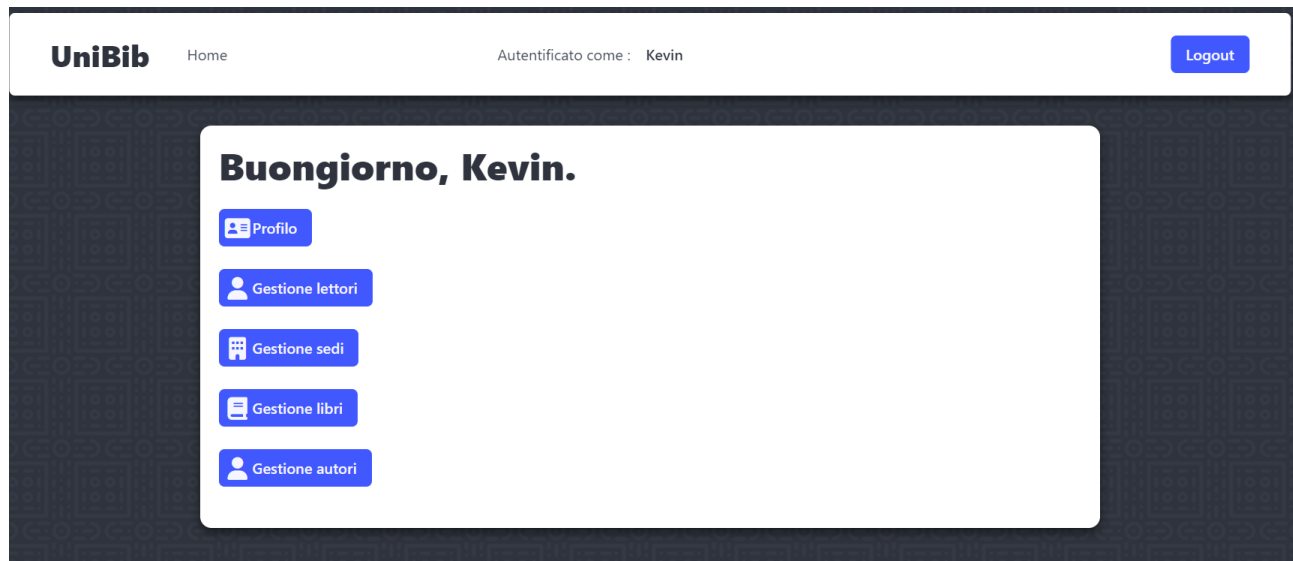
Archivio Prestiti

ISBN	Data Inizio	Data Fine
IF200S04	2024-12-20	2025-01-20

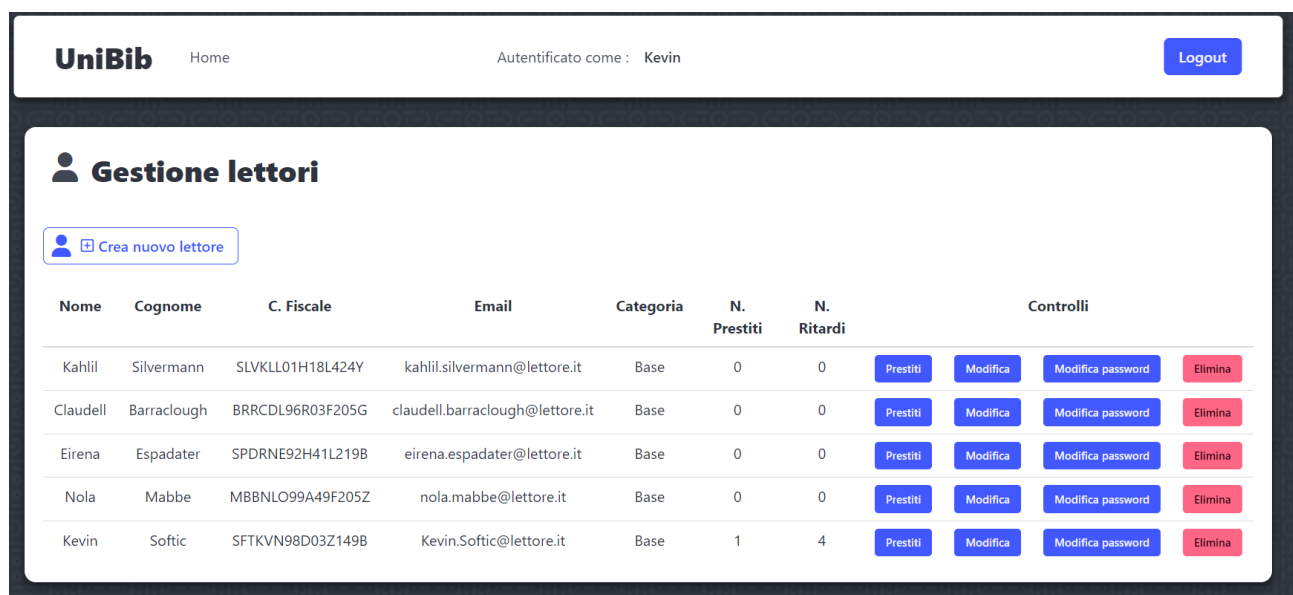
Usciamo con il tasto di logout presente nella navbar.

Bibliotecario

Loggandosi, invece, come bibliotecario possiamo notare come ci siano delle differenze nelle opzioni proposte:



L'opzione "Profilo" è analoga a quella di lettore, con le dovute differenze, quindi concentriamoci sulle opzioni restanti. Nell'opzione "Gestione lettori" verrà data una tabella con tutti i lettori registrati nel sistema con delle informazioni generali e con 5 possibilità di gestione: creare un nuovo lettore, gestire i prestiti di un lettore, modificare i dati di un lettore, modificare la password di un lettore ed eliminare un lettore.



Nella gestione dei prestiti di un lettore ci sono due possibilità: quella di modifica, cioè quello della proroga del prestito, oppure quello di consegna:

UniBib

Home

Autenticato come : Kevin

Logout

Gestione prestiti

Cod. Libro	C. Fiscale	Periodo Ammesso	Controlli	
IF200S04	SFTKVN98D03Z149B	2024-12-20 - 2025-01-20	Modifica	Consegna

Nella modifica del lettore invece è possibile modificare l’email, il nome, il cognome, la categoria di lettore ed il suo numero di ritardi:

UniBib

Home

Autenticato come : Kevin

Logout

Modifica lettore

Email

Kevin.Softic@lettore.it

Nome

Kevin

Cognome

Softic

Categoria

Base

Ritardi

0

Modifica lettore

Nella “Gestione sedi” verrà mostrata una tabella contenente tutte le sedi della biblioteca con il loro indirizzo e con 4 possibilità d’interazione: la creazione di una nuova sede, le statistiche, il report sui ritardi e la modifica delle sedi.

UniBib
Home
Autenticato come : Kevin
Logout

Gestione Sedi

Crea nuova sede

ID	Città	Via	Controlli		
972890e9-e50b-4c40-8daa-c2a3921aa464	Milano	Via Celoria 18	Statistiche	Report ritardi	Modifica
9d720142-26b5-4f6f-9d6d-b880e9d6c469	Milano	Corso Italia 20	Statistiche	Report ritardi	Modifica

Le statistiche di una sede mostrano il numero di libri, di copie e di prestiti in corso gestiti dalla sede:

UniBib
Home
Autenticato come : Kevin
Logout

Statistica sede

Città	Via	# Copie	# Libri	# Prestiti
Milano	Corso Italia 20	3	2	0

Mentre il report sui ritardi di una sede mostra una lista di prestiti correntemente in ritardo elencando libro e lettore:

UniBib
Home
Autenticato come : Kevin
Logout

Report sede

Città	Via	ISBN	Titolo	Nome	Cognome
Milano	Via Celoria 18	IF200S04	Quarta Prova	Kevin	Softic

Selezionando “Gestione Libri” otterremmo una lista di copie presenti nel sistema con i vari dettagli: ISBN, id, Titolo, città, indirizzo, disponibilità ed autori. Oltre alle informazioni il bibliotecario avrà la possibilità di: creare un nuovo libro, aggiungere una copia di un libro, modificare i dati di un libro, modificare la sede di una copia, eliminare una copia oppure eliminare un libro e tutte le sue copie.

UniBib

Home

Autenticato come : Kevin

Logout

Gestione Libri

Aggiungi nuovo libro

Aggiungi nuova copia

ISBN	Id	Titolo	Città	Indirizzo	Disponibilità	Autori	Controlli			
IF200S02	6a3a8668-ef32-4250-ad3f-a15fa21a8766	Seconda Prova	Milano	Corso Italia 20	f	Dante Alighieri, Dante Aligo	Modifica	Modifica Sede	Elimina Copia	Elimina libro
IF200S02	6afcb87b-88bb-4a08-898a-67826e934bbe	Seconda Prova	Milano	Via Celoria 18	t	Dante Aligo, Dante Alighieri	Modifica	Modifica Sede	Elimina Copia	Elimina libro
IF200S02	bae36e43-8e6c-476f-a2f7-da522b5d6803	Seconda Prova	Milano	Corso Italia 20	t	Dante Alighieri, Dante Aligo	Modifica	Modifica Sede	Elimina Copia	Elimina libro
IF200S03	9e40adb8-b9e5-45a1-b9fe-763bec303d83	Terza Prova	Milano	Corso Italia 20	f	Dante Alighieri, Dante Aligo	Modifica	Modifica Sede	Elimina Copia	Elimina libro
IF200S04	6ba2367c-b14f-48e7-9085-6fae228ee35e	Quarta Prova	Milano	Via Celoria 18	f	Dante Aligo, Dante Alighieri	Modifica	Modifica Sede	Elimina Copia	Elimina libro

Infine, scegliendo “Gestione Autori” si ottiene una lista con tutti gli autori registrati nel sistema, con inoltre la possibilità di creare un nuovo autore, modificare un autore oppure di eliminarlo:

UniBib

Home

Autenticato come : Kevin

Logout

Gestione Autori

Aggiungi nuovo autore

Nome	Cognome	Data Nascita	Data Morte	Controlli	
Dante	Aligo	1500-10-15	2024-08-01	Modifica	Elimina
Dante	Alighieri	1457-08-03	2024-08-07	Modifica	Elimina